

Task 8:

Exploiting SQL Injection Vulnerability - DVWA (Medium Security)

Understanding the Setup

To analyze and exploit SQL Injection vulnerabilities in DVWA (Damn Vulnerable Web Application) with Medium Security level, the following setup was performed:

1. Logged into DVWA and navigated to the SQL Injection page.
2. Set the security level to Medium (DVWA Security → Medium).
3. Observed the web interface and the form structure for SQL Injection testing.

The SQL Injection page was examined, revealing that the form uses a POST request with the following structure:

```
<form action="#" method="POST">
  <p>
    " User ID: "
    <select name="id"> == $0
      <option value="1">1</option>
      <option value="2">2</option>
      <option value="3">3</option>
      <option value="4">4</option>
      <option value="5">5</option>
    </select>
    <input type="submit" name="Submit" value="Submit">
  </p>
</form>
```

Task: Display Database Version and Database Name Together

The objective was to exploit an SQL Injection vulnerability under Medium security to retrieve both the database version and database name in a single request.

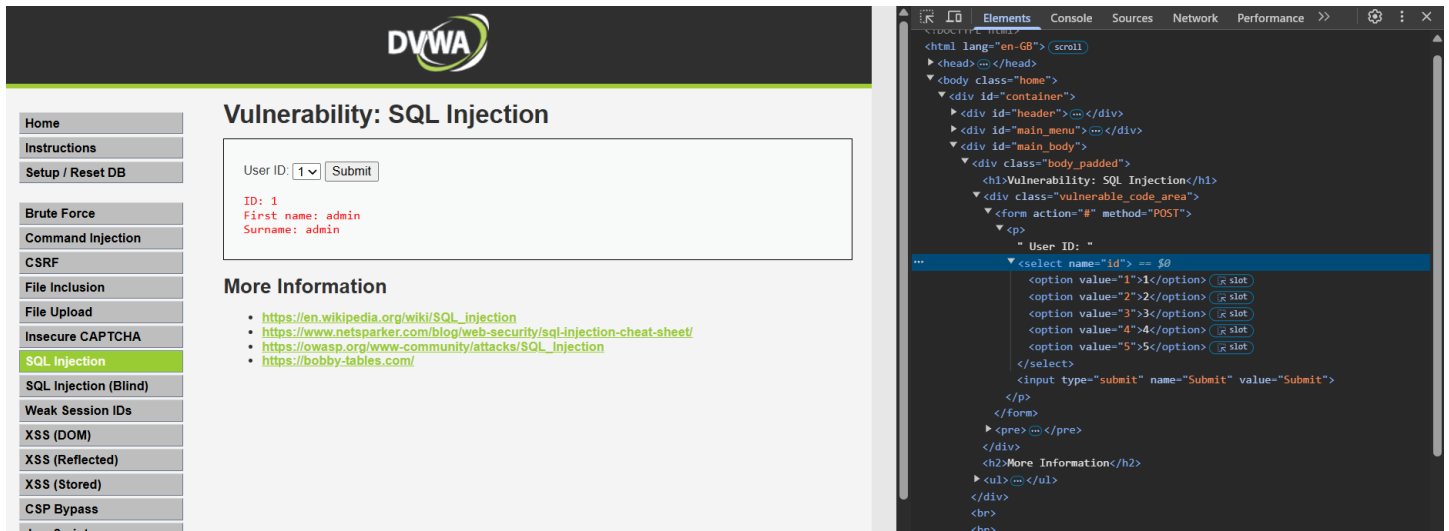
Solutions Used

I performed the solution using two different approaches:

Solution 1: Using Developer Tools (DevTools)

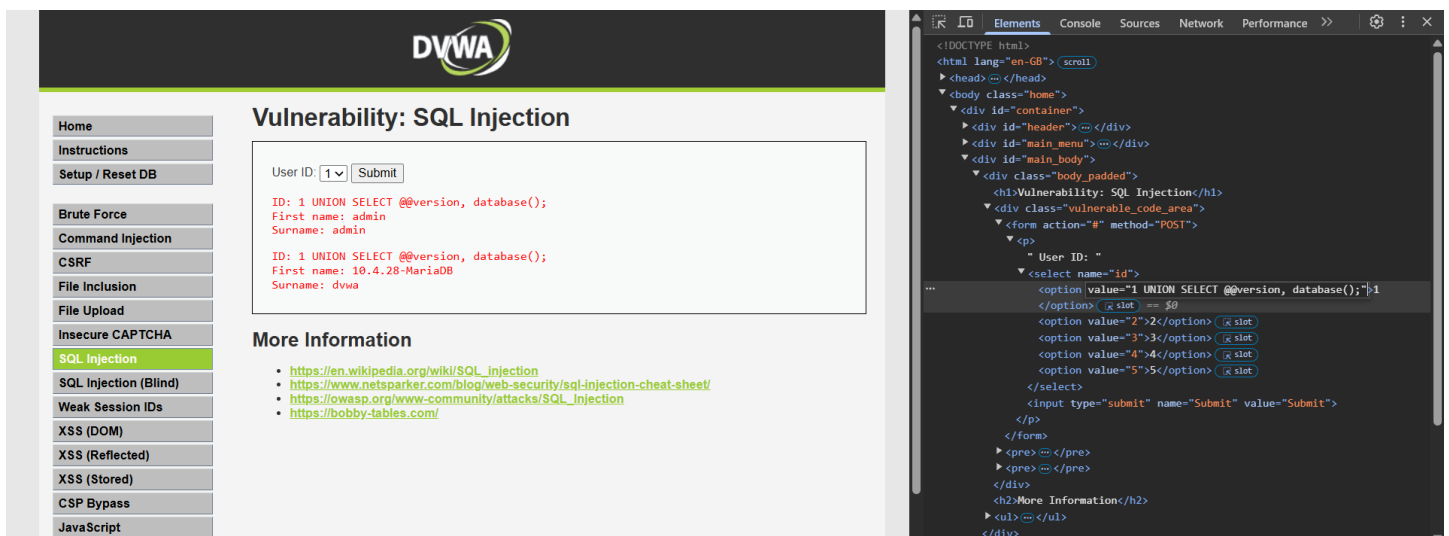
1. Inspecting the Form

- Right-clicked on the form and selected "Inspect" in the browser to examine the HTML structure.
- Identified that user input is directly passed via the id parameter in a POST request.



2. Performing SQL Injection

- Tested basic SQL injection payloads to check if the input field is vulnerable.
- Used the following payload to extract both database version and name:



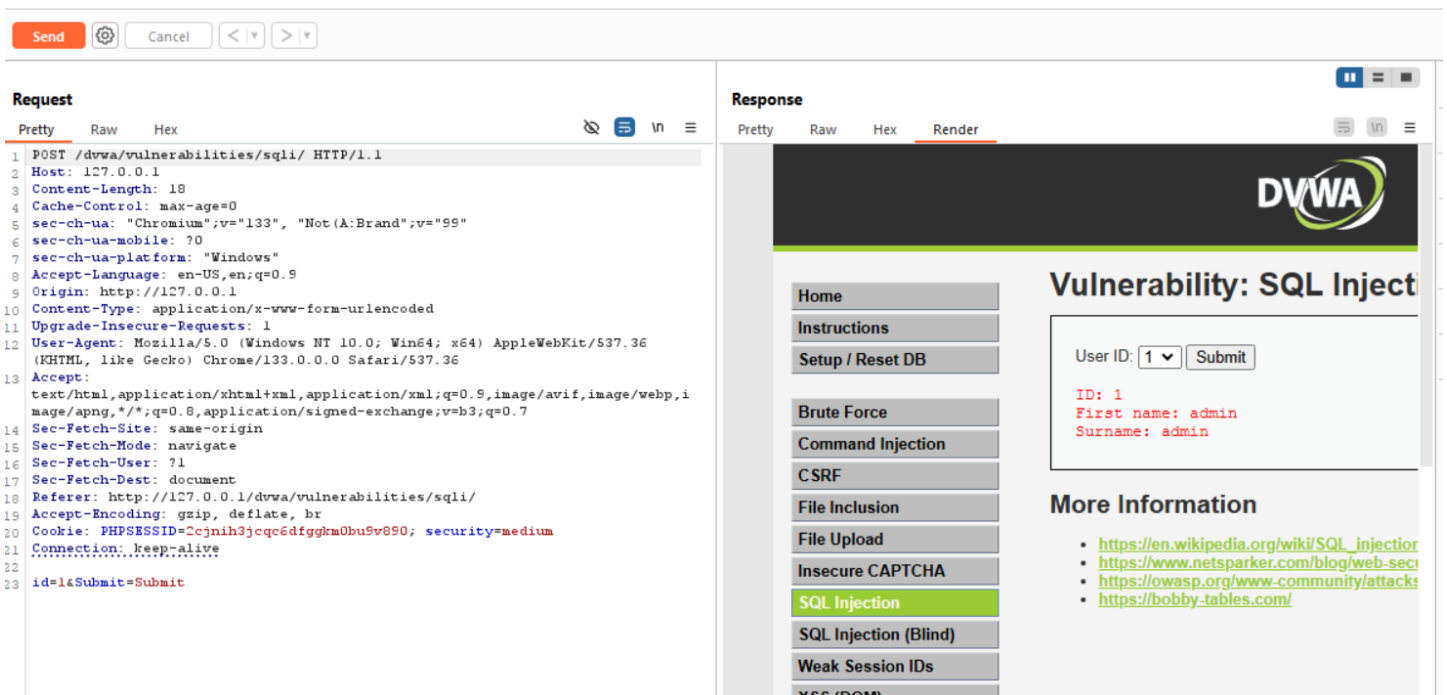
3. 1' UNION SELECT database(), version() -- -

- This query successfully displayed both the database name and version on the page.

Solution 2: Using Burp Suite

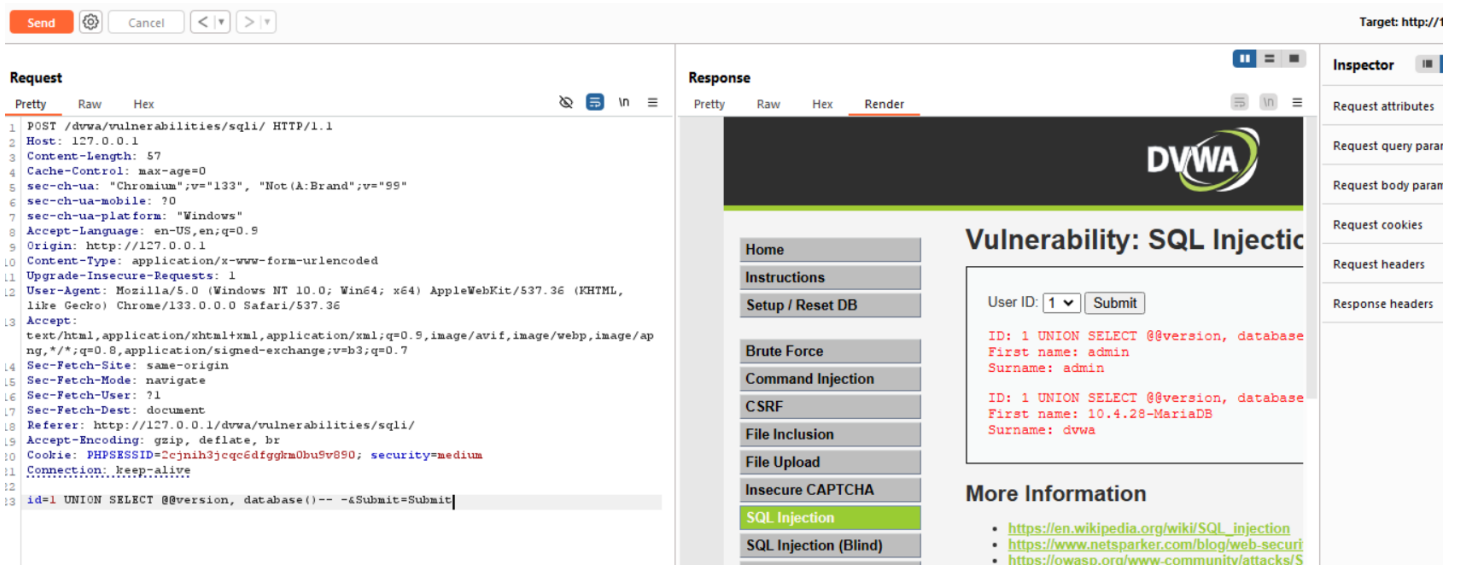
1. Intercepting the Request

- Used Burp Suite to intercept the form submission request.
- Captured the POST request and modified the id parameter manually.



2. Injecting SQL Payload

- Replaced the id parameter value with:



3. 1' UNION SELECT database(), version() -- -

- Forwarded the request and observed the response.
- Successfully retrieved both the database name and version in the response body.

State of Code After SQL Injection

Before SQL Injection, the original query was:

\$query = "SELECT first_name, last_name FROM users WHERE user_id = \$id;";

After injecting the payload 1' UNION SELECT database(), version() -- -, the executed query becomes:

\$query = "SELECT first_name, last_name FROM users WHERE user_id = 1 UNION SELECT database(), version() -- -;"

This modification allows the attacker to retrieve the database name and version instead of the expected user information.